

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОНИКИ И МАТЕМАТИКИ
им. А.Н. Тихонова НИУ ВШЭ

ДЕПАРТАМЕНТ КОМПЬЮТЕРНОЙ ИНЖЕНЕРИИ

КУРСОВАЯ РАБОТА
по дисциплине «Алгоритмизация и программирование»

Тема: «Разработка приложения для пакетной конвертации видеороликов в
эффективный формат хранения»

Выполнил:
студент группы БИВ253
Солодилов Михаил Анатольевич

Руководитель:
Волков Владимир Олегович

Москва, 2026

Задание на курсовую работу

Тема: разработка программы для пакетного преобразования видеозаписей в более компактный формат, позволяющий уменьшить объем занимаемого места на носителе.

Цель работы: разработать приложение, с помощью которого пользователь сможет выбрать один или несколько видеофайлов и перекодировать их в формат HEVC, задав параметры качества, разрешения и частоты кадров.

Задачи работы:

- 1) изучить предметную область хранения и преобразования видеофайлов;
- 2) рассмотреть существующие средства и методы сжатия видео;
- 3) выбрать технологический стек для разработки десктопного приложения;
- 4) реализовать графический интерфейс для выбора файлов и настройки параметров перекодирования;
- 5) реализовать модуль перекодирования с использованием FFmpeg;
- 6) добавить пакетную обработку файлов и отображение хода выполнения;
- 7) протестировать приложение;
- 8) подготовить пользовательскую и разработческую документацию.

Функциональные требования:

- 1) программа должна позволять выбирать один или несколько видеофайлов через стандартный диалог выбора файлов;
- 2) программа должна поддерживать добавление файлов перетаскиванием в окно приложения;
- 3) программа должна работать с локальными видеофайлами, доступными файловой системе пользователя;
- 4) программа должна позволять задавать параметр качества перекодирования;
- 5) программа должна позволять ограничивать высоту выходного видео;

- 6) программа должна позволять ограничивать частоту кадров выходного видео;
- 7) программа должна создавать выходной файл с суффиксом `_compressed`;
- 8) программа должна позволять копировать метаданные исходного файла в результат;
- 9) программа должна отображать прогресс выполнения операции;
- 10) программа должна сообщать пользователю об ошибке, если перекодирование завершилось неуспешно.

Нефункциональные требования:

- 1) язык реализации: Python;
- 2) графический интерфейс: GTK 4 через PyGObject;
- 3) обработка видео: FFmpeg и библиотека python-ffmpeg;
- 4) копирование метаданных: FFmpeg и ExifTool;
- 5) платформа: десктопная операционная система Linux с установленными FFmpeg и GTK 4;
- 6) формат хранения данных: локальные файлы пользователя, без использования базы данных.

Ожидаемые результаты:

- 1) рабочее приложение для пакетного перекодирования видео;
- 2) исходный код проекта;
- 3) инструкция по запуску и использованию;
- 4) примеры входных данных и результата перекодирования.

График выполнения

№	Этап работы	Дата
1	Обоснование темы, описание предметной области, постановка цели и задач	29.11.2025
2	Краткий обзор аналогов и источников литературы	13.12.2025
3	Подготовка окончательной версии технического задания	20.12.2025
4	Реализация базового модуля перекодирования через FFmpeg	08.05.2026
5	Реализация графического интерфейса и пакетной обработки файлов	08.05.2026
6	Тестирование приложения и подготовка документации	15.05.2026

Аннотация

В курсовой работе рассматривается разработка десктопного приложения Floppy для пакетной конвертации видеороликов в формат HEVC. Объектом разработки является программное средство, предназначенное для уменьшения объема, занимаемого видеофайлами на диске. Цель работы состоит в создании приложения с графическим интерфейсом, которое позволяет выбрать несколько видеофайлов, настроить качество, разрешение и частоту кадров, а затем выполнить перекодирование для экономии места на носителе.

В ходе работы были рассмотрены основные подходы к сжатию видео, выбран стек Python, GTK 4, PyGObject, FFmpeg, ExifTool и python-ffmpeg. В приложении реализованы модули пользовательского интерфейса, выбора параметров, определения доступного HEVC-кодировщика, запуска FFmpeg, копирования метаданных и отображения прогресса. В результате была создана программа для локальной обработки видеофайлов без использования сетевых сервисов и базы данных.

Работа содержит 20 страниц, 1 иллюстрацию, 1 таблицу, 8 использованных источников.

ОГЛАВЛЕНИЕ

1	Введение	7
2	Обзор аналогов и источников	8
3	Описание разработанного приложения	9
4	Архитектура и реализация	10
4.1	Графический интерфейс	10
4.2	Пакетная обработка	10
4.3	Модуль перекодирования	11
4.4	Выбор кодировщика	11
4.5	Копирование метаданных	12
5	Ход разработки	13
6	Тестирование	14
7	Заключение	15
8	Список использованных источников	16
A	Руководство пользователя	17
A.1	Назначение программы	17
A.2	Установка	17
A.3	Запуск	17
A.4	Основной сценарий использования	17
A.5	Типовые ошибки	18
B	Руководство разработчика	19
B.1	Структура проекта	19
B.2	Поток выполнения	19
B.3	Выбор кодировщика	19
B.4	Сборка и запуск при разработке	20
B.5	Возможные направления развития	20

1 Введение

Видеофайлы, снятые на мобильные устройства, камеры или полученные в результате записи экрана, часто занимают значительный объем дискового пространства. Это особенно заметно при хранении архивов, пересылке файлов или накоплении большого количества коротких роликов. Если видео не планируется использовать для дальнейшего монтажа, его можно перекодировать с другими параметрами качества, разрешения и частоты кадров, тем самым уменьшив размер файла.

Один из распространенных способов уменьшения размера видео заключается в переходе на более эффективный видеокодек. В данной работе используется HEVC, также известный как H.265. В описании рекомендации ITU-T указано, что стандарт был разработан в ответ на потребность в более высоком сжатии движущихся изображений [1]. Для обработки видео выбран FFmpeg. Его документация описывает инструмент как универсальный медиаконвертер, который может читать различные входные данные, фильтровать и транскодировать их в разные выходные форматы [2].

Актуальность темы связана с тем, что работа с FFmpeg через командную строку требует от пользователя знания кодеков, параметров качества, фильтров и правил формирования выходных файлов. В приложении Florpy эти действия перенесены в графический интерфейс. Пользователь задает параметры через элементы управления, а приложение самостоятельно формирует команду и выбирает доступный HEVC-кодировщик.

Целью курсовой работы является разработка приложения Florpy для пакетной конвертации видеороликов в эффективный формат хранения. Для достижения этой цели необходимо решить следующие задачи:

- определить требования к приложению;
- изучить инструменты перекодирования видео;
- реализовать запуск FFmpeg из Python-приложения;
- реализовать выбор файлов и параметров обработки;
- обеспечить отображение прогресса и сообщений о результате;
- провести тестирование основных пользовательских сценариев.

2 Обзор аналогов и источников

Для уменьшения размера видео можно использовать как консольные инструменты, так и готовые графические приложения. В рамках данной работы были рассмотрены решения, которые позволяют выполнять перекодирование видео и настраивать параметры результата.

В качестве базового инструмента обработки видео выбран FFmpeg. Его документация описывает поддержку входных файлов, фильтрации и транскодирования в выходные форматы [2]. Для приложения Floppy это важно, поскольку собственная реализация видеокодека не требуется: программа только формирует параметры обработки и запускает FFmpeg.

В качестве графического аналога рассмотрен HandBrake. Документация HandBrake описывает готовые профили настроек для устройств и типовых сценариев, а также аппаратные профили для кодирования через специализированные медиаблоки современных видеоустройств [3]. В HandBrake также есть функция *Passthru common Metadata*, однако документация описывает ее как перенос ограниченного набора распространенных полей метаданных [4]. В Floppy выбран другой сценарий: при включении *Copy metadata* приложение сначала копирует метаданные и главы средствами FFmpeg, а затем переносит теги через ExifTool с сохранением групп тегов [5, 6].

Графический интерфейс Floppy реализован на GTK 4 через PyGObject. GTK предоставляет необходимые виджеты для окон, кнопок, полей ввода, диалогов выбора файлов и индикаторов прогресса [7]. Для асинхронного запуска FFmpeg из Python выбрана библиотека *python-ffmpeg*, которая позволяет управлять обработкой из кода и получать события прогресса [8].

По итогам анализа был выбран подход, при котором пользователь взаимодействует с графическим интерфейсом, а вся обработка видео выполняется через FFmpeg. Такой вариант соответствует задаче курсовой работы: разработать удобную оболочку для пакетного перекодирования без самостоятельной реализации алгоритмов видеосжатия.

3 Описание разработанного приложения

Разработанное приложение Floppy представляет собой десктопную программу с графическим интерфейсом. Оно предназначено для пакетного перекодирования видеороликов в HEVC. Пользователь выбирает один или несколько файлов, задает параметры обработки и запускает кодирование. Для каждого входного файла рядом с ним создается отдельный выходной файл с суффиксом `_compressed`.

Проект состоит из следующих основных файлов:

- `app.py` — модуль графического интерфейса и пользовательских сценариев;
- `engine.py` — модуль запуска перекодирования;
- `utils.py` — вспомогательный модуль для работы с FFmpeg, FFprobe и параметрами кодировщиков;
- `requirements.txt` — список Python-зависимостей проекта.

Пользовательский интерфейс реализован в классе `MainWindow`. Главное окно содержит название приложения, строку с выбранными файлами, кнопку выбора файлов, подсказку о поддержке перетаскивания, поля настройки качества, разрешения и частоты кадров, переключатель копирования метаданных, индикатор прогресса, строку статуса и кнопку запуска перекодирования.

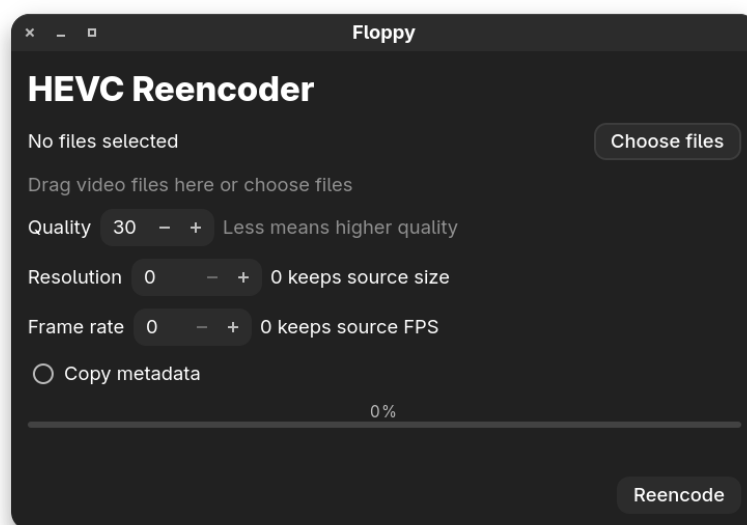


Рис. 1: Главное окно приложения Floppy

4 Архитектура и реализация

4.1 Графический интерфейс

Модуль `app.py` создает GTK-приложение с идентификатором `dev.floppy.Reencoder`. При активации приложения создается главное окно. Элементы интерфейса расположены вертикально: в верхней части находится заголовок, ниже размещены элементы выбора файлов и настройки параметров, затем индикатор прогресса и кнопка запуска.

Выбор файлов реализован через `Gtk.FileChooserNative`. Диалог поддерживает множественный выбор, поэтому пользователь может сразу добавить несколько видеофайлов. Кроме того, в окно добавлен обработчик перетаскивания файлов через `Gtk.DropTarget`. Благодаря этому файлы можно добавить как через стандартный диалог, так и простым переносом в окно приложения.

Для настройки качества используется числовое поле со значениями от 1 до 51. Разрешение и частота кадров также задаются числовыми полями. Значение 0 для разрешения означает сохранение исходного размера кадра, а значение 0 для частоты кадров — сохранение исходной частоты. Переключатель `Copy metadata` включает перенос метаданных из исходного файла в выходной.

4.2 Пакетная обработка

После нажатия кнопки `Reencode` приложение проверяет, выбраны ли файлы. Если список пуст, пользователь получает сообщение `Choose files first`. Если файлы выбраны, интерфейс считывает параметры качества, разрешения, частоты кадров и состояние переключателя копирования метаданных. Затем кнопки выбора и запуска временно блокируются, чтобы пользователь не мог начать вторую обработку параллельно с уже запущенной.

Перекодирование выполняется в отдельном потоке, поэтому основной поток GTK не блокируется. Во время обработки пользователь видит полосу прогресса, название текущего файла и количество уже обработанных файлов. Если возникает ошибка, приложение останавливает обработку, разблокирует элементы управления и показывает сообщение с информацией о том, сколько файлов было успешно обработано до сбоя.

4.3 Модуль перекодирования

Модуль `engine.py` содержит асинхронную функцию `reencode`. Она принимает путь к входному файлу, параметр качества, ограничение разрешения, ограничение частоты кадров, признак копирования метаданных и необязательную функцию обратного вызова для обновления прогресса.

Перед запуском FFmpeg модуль определяет исходное разрешение и частоту кадров. Если пользовательское ограничение выше или равно исходному значению, соответствующий параметр не применяется. Это исключает бессмысленное увеличение разрешения или частоты кадров при задаче сжатия. Если включено копирование метаданных, модуль заранее проверяет доступность ExifTool, чтобы не запускать длительное перекодирование при отсутствии необходимой внешней утилиты.

Имя выходного файла формируется на основе имени исходного файла: к нему добавляется суффикс `_compressed`, а расширение сохраняется. Далее создается объект FFmpeg, указываются входной и выходной файлы, а также рассчитанные параметры кодирования. При включенном копировании метаданных в параметры FFmpeg добавляются `map_metadata=0` и `map_chapters=0`. После завершения кодирования вызывается ExifTool с аргументами `-TagsFromFile, -all:all` и путем к выходному файлу.

4.4 Выбор кодировщика

Модуль `utils.py` отвечает за выбор доступного HEVC-кодировщика. В проекте описаны варианты `hevc_nvenc`, `hevc_qsv`, `hevc_vaapi`, `hevc_amf`, `hevc_vulkan` и `libx265`. Сначала программа получает список кодировщиков, доступных в установленной сборке FFmpeg, затем проверяет их в порядке заданного приоритета.

Если найден подходящий аппаратный кодировщик, приложение может использовать его для ускорения обработки. Если аппаратный вариант недоступен, используется программный кодировщик `libx265`. Для каждого кодировщика заданы собственные параметры качества, поскольку разные реализации FFmpeg используют разные имена и смысл опций.

Такой подход делает приложение более переносимым между компьютерами с разными видеокартами и разными сборками FFmpeg. Пользователю при этом не нужно вручную выбирать конкретный кодировщик или разбираться в особенностях

своей системы.

4.5 Копирование метаданных

Копирование метаданных реализовано как необязательный режим. Если пользователь включает `Copy metadata`, приложение выполняет две операции. Сначала FFmpeg получает параметры `map_metadata=0` и `map_chapters=0`, которые указывают копировать метаданные и главы из первого входного файла [5]. Затем, после успешного перекодирования, вызывается ExifTool с командой копирования тегов из исходного файла в выходной. Документация ExifTool описывает форму `-TagsFromFile src -all:all dst` как копирование тегов с сохранением исходных групп [6].

Если ExifTool недоступен, приложение завершает операцию до запуска FFmpeg и выводит понятное сообщение об ошибке. Это позволяет избежать ситуации, при которой пользователь ждет завершения перекодирования, а затем узнает, что метаданные не могут быть перенесены.

5 Ход разработки

Разработка проекта велась поэтапно. Сначала была сформулирована основная задача: создать программу для пакетной конвертации видеороликов в более эффективный формат хранения. Для реализации был выбран стек Python и FFmpeg. Python используется для графического приложения и связующей логики, а FFmpeg выполняет непосредственную обработку видео.

На первом этапе был реализован базовый модуль перекодирования одного файла. Он запускал FFmpeg, формировал имя выходного файла и применял параметр качества. Затем в проект была добавлена работа с FFprobe, с помощью которого приложение получает технические сведения об исходном видео: разрешение, частоту кадров и другие параметры, необходимые для корректной настройки обработки.

Следующим этапом стала реализация выбора кодировщика. Приложение проверяет доступные HEVC-кодировщики и выбирает подходящий вариант из заданного списка приоритетов. Это позволяет использовать аппаратное кодирование там, где оно доступно, и переходить на программное кодирование через `libx265`, если аппаратный кодировщик отсутствует.

После этого был разработан графический интерфейс на GTK 4. В него были добавлены выбор файлов, настройки качества, разрешения и частоты кадров, а также индикатор прогресса. Затем была реализована обработка нескольких файлов и добавление файлов перетаскиванием. На заключительном этапе в приложение был добавлен режим копирования метаданных.

6 Тестирование

Для проверки приложения были выделены основные контрольные сценарии, связанные с функциональными требованиями.

- 1) **Запуск приложения.** Программа должна открыть главное окно, отобразить элементы управления и не завершиться с ошибкой при наличии необходимых зависимостей.
- 2) **Выбор одного файла.** После выбора одного видео в интерфейсе должно отображаться имя файла, а кнопка запуска должна начинать обработку.
- 3) **Выбор нескольких файлов.** После выбора нескольких файлов приложение должно отображать их количество и последовательно обрабатывать список.
- 4) **Перетаскивание файлов.** Файлы должны добавляться переносом в окно приложения.
- 5) **Настройка качества.** Указанное пользователем значение качества должно передаваться в параметры выбранного кодировщика.
- 6) **Ограничение разрешения.** При значении 0 должен сохраняться исходный размер кадра, а при меньшем значении должен применяться фильтр масштабирования.
- 7) **Ограничение частоты кадров.** При значении 0 должна сохраняться исходная частота кадров, а при меньшем значении должен применяться фильтр fps.
- 8) **Копирование метаданных.** При включенном переключателе Copy metadata должны добавляться параметры копирования метаданных FFmpeg и выполняться перенос тегов через ExifTool.
- 9) **Обработка ошибки.** При ошибке FFmpeg приложение должно разблокировать кнопки и вывести сообщение о сбое.

Эти сценарии покрывают основные функции, указанные в техническом задании: выбор файлов, настройку параметров, запуск перекодирования, копирование метаданных, отображение прогресса, обработку ошибок и сохранение результата в файл с суффиксом `_compressed`.

7 Заключение

В ходе курсовой работы было разработано десктопное приложение Floppy для пакетной конвертации видеороликов в эффективный формат хранения. Программа реализована на языке Python с использованием GTK 4 для графического интерфейса и FFmpeg для обработки видео.

В приложении реализованы выбор одного или нескольких файлов, добавление файлов перетаскиванием, настройка качества, ограничение разрешения и частоты кадров, копирование метаданных, последовательное перекодирование файлов, отображение прогресса и сообщений о результате. Для повышения переносимости добавлен автоматический выбор доступного HEVC-кодировщика.

Разработанное приложение решает поставленную задачу: оно позволяет уменьшать размер видеофайлов без ручного ввода команд FFmpeg. В дальнейшем проект можно развить, добавив выбор каталога для результатов, журнал выполненных операций, отмену текущей обработки и предварительную оценку ожидаемого размера выходного файла.

8 Список использованных источников

- [1] ITU-T. *Recommendation H.265: High efficiency video coding*. URL: <https://www.itu.int/rec/T-REC-H.265> (дата обр. 08.05.2026).
- [2] FFmpeg. *ffmpeg Documentation*. URL: <https://ffmpeg.org/ffmpeg.html> (дата обр. 08.05.2026).
- [3] HandBrake. *Official presets*. URL: <https://handbrake.fr/docs/en/latest/technical/official-presets.html> (дата обр. 08.05.2026).
- [4] HandBrake. *Metadata Passthru*. URL: <https://handbrake.fr/docs/en/latest/advanced/metadata-passthru.html> (дата обр. 09.05.2026).
- [5] FFmpeg. *Metadata options*. URL: <https://ffmpeg.org/ffmpeg-doc.html> (дата обр. 09.05.2026).
- [6] ExifTool. *ExifTool application documentation*. URL: https://exiftool.org/exiftool_pod2.html (дата обр. 09.05.2026).
- [7] PyGObject. *PyGObject Documentation*. URL: <https://pygobject.gnome.org/> (дата обр. 09.05.2026).
- [8] python-ffmpeg. *python-ffmpeg Documentation*. URL: <https://python-ffmpeg.readthedocs.io/> (дата обр. 08.05.2026).

А Руководство пользователя

А.1 Назначение программы

Florpy предназначено для пакетного перекодирования видеороликов в HEVC. Программа помогает уменьшить размер видеофайлов на диске и не требует от пользователя ручного ввода команд FFmpeg.

А.2 Установка

Для работы приложения требуются Linux, Python, GTK 4, FFmpeg, ExifTool и Python-зависимости из файла `requirements.txt`. Перед запуском необходимо установить системные пакеты GTK 4, FFmpeg и ExifTool средствами пакетного менеджера операционной системы, а затем установить зависимости Python.

Пример установки Python-зависимостей:

```
pip install -r requirements.txt
```

А.3 Запуск

Запуск приложения выполняется из каталога проекта:

```
python app.py
```

После запуска открывается окно Florpy с элементами выбора файлов и настройки параметров.

А.4 Основной сценарий использования

- 1) нажать кнопку `Choose files` или перетащить видеофайлы в окно приложения;
- 2) выбрать значение качества;
- 3) при необходимости указать максимальную высоту видео;
- 4) при необходимости указать максимальную частоту кадров;
- 5) при необходимости включить `Copy metadata`;

- 6) нажать кнопку Reencode;
- 7) дождаться завершения обработки;
- 8) найти рядом с исходным файлом новый файл с суффиксом `_compressed`.

A.5 Типовые ошибки

Если пользователь нажал Reencode, но не выбрал файлы, программа выводит сообщение `Choose files first`. В этом случае нужно выбрать хотя бы один видеофайл и повторить запуск.

Если в системе не установлен FFmpeg или отсутствует подходящий HEVC-кодировщик, перекодирование завершится ошибкой. Для решения проблемы нужно установить FFmpeg или использовать сборку FFmpeg с поддержкой HEVC-кодирования.

Если включен режим `Copy metadata`, но в системе не установлен ExifTool, приложение сообщит об ошибке до запуска перекодирования. В этом случае нужно установить ExifTool или отключить копирование метаданных.

Если выбранный файл поврежден или не является видеофайлом, FFmpeg может завершить обработку с ошибкой. В таком случае следует проверить исходный файл или выбрать другой файл.

В Руководство разработчика

В.1 Структура проекта

Проект Floppy расположен в каталоге `floppy`. Основные файлы:

- `app.py` — точка входа приложения и описание графического интерфейса;
- `engine.py` — логика перекодирования одного файла;
- `utils.py` — работа с FFmpeg, FFprobe, ExifTool, кодировщиками и параметрами вывода;
- `requirements.txt` — список Python-зависимостей.

В.2 Поток выполнения

При запуске `app.py` создается объект `FloppyApp`, а затем главное окно `MainWindow`. Пользователь выбирает файлы, после чего обработчик кнопки считывает значения из интерфейса и запускает отдельный поток. В этом потоке для каждого файла вызывается асинхронная функция `engine.reencode`.

Функция `reencode` получает технические данные исходного видео через функции из `utils.py`, формирует параметры FFmpeg и запускает обработку. Если включено копирование метаданных, перед обработкой проверяется наличие ExifTool, а после успешного завершения FFmpeg выполняется перенос тегов в выходной файл. Прогресс передается обратно в интерфейс через `callback`. Обновление GTK-виджетов выполняется через `GLib.idle_add`, чтобы изменения происходили в основном потоке интерфейса.

В.3 Выбор кодировщика

Доступные кодировщики описаны в словаре `ENCODERS`. Порядок выбора задан списком `ENCODER_PRIORITIES`. Для добавления нового кодировщика нужно:

- добавить описание в `ENCODERS`;
- указать имя кодека FFmpeg;

- задать параметры качества;
- при необходимости указать аппаратное ускорение и дополнительные параметры;
- добавить ключ кодировщика в `ENCODER_PRIORITIES`.

В.4 Сборка и запуск при разработке

Перед разработкой нужно установить зависимости:

```
pip install -r requirements.txt
```

Также в системе должны быть доступны исполняемые файлы `ffmpeg`, `ffprobe` и `exiftool`. Проверить их наличие можно командами:

```
ffmpeg -version  
ffprobe -version  
exiftool -ver
```

Запуск приложения для проверки:

```
python app.py
```

В.5 Возможные направления развития

В дальнейшем в приложение можно добавить выбор каталога для результатов, отображение подробного журнала FFmpeg, отмену текущей операции и предварительную оценку ожидаемого размера файла.